

Program 7

```

#include <stdio.h>
#include <stdlib.h>
#include <stdlib.h>
int main()
{
    float weight[50], profit[50], ratio[50], Total_value,
    temp, capacity, amount;
    int n, i, j;
    printf("Enter the number of items:");
    scanf("%d", &n);
    for(i = 0; i < n; i++)
    {
        printf("Enter the weight and profit for item (%d): \n", i);
        scanf("%f %f", &weight[i], &profit[i]);
    }
    printf("Enter the capacity of knapsack: \n");
    scanf("%f", &capacity);
    for(i = 0; i < n; i++)
        ratio[i] = profit[i] / weight[i];
    for(i = 0; i < n; i++)
        for(j = i + 1; j < n; j++)

```


Output

Enter the amount of items : 3

Enter the weight & profit for item[0] : 100 3

Enter the weight & profit for item[1] : 98 14

Enter the weight & profit for item[2] : 17 16

Enter the capacity of knapsack : 111

Knapsack problem using greedy algorithm

the max value is : 19.52

$$\{ \text{if}(\text{ratio}[i] < \text{ratio}[j])$$

```
temp = ratio[j];  
ratio[j] = ratio[i];  
ratio[i] = temp;
```

```
temp = weight[j];  
weight[j] = weight[i];  
weight[i] = temp;
```

```
temp = profit[j];
profit[j] = profit[i];
profit[i] = temp;
```

printf ("knapsack problems using greedy algo"
-thm : 10");

$$\{ \text{for } (i=0; i < n; i++)$$

$!P(\text{weight}[i] > \text{capacity})$

break;

else

f

Total value = Total value + profit [r];

Capacity = capacity - weight[i];


```
}  
}  
if (p < n)  
    total value = total value + (ratio[i] * capacity);  
    printf ("In The max value is : %d\n", total  
        value);  
    return 0;  
}
```

Output

Enter the size of set: 9

Enter the set in increasing order:

1 2 3 4 5 6 7 8 9

Enter the value of d: 9

subset: 1 2 6

subset: 1 3 5

subset: 1 8

subset: 2 3 4

subset: 2 7

subset: 3 6

subset: 4 5

subset: 9

Program 8

```
#include <stdio.h>
#include <conio.h>
int s[10], x[10], d;
void SumOfSub (int, int, int);
int main()
{
    int n, sum=0;
    int i;
    printf("\n Enter the size of the set:");
    scanf("%d", &n);
    printf("\n Enter the set in increasing order:
    \n");
    for (i=1; i<=n; i++)
        scanf("%d", &s[i]);
    printf("\n Enter the value of d:\n");
    scanf("%d", &d);
    for (i=1; i<=n; i++)
        sum = sum + s[i];
    if (sum < d || s[i] > d)
        printf("\n No subset possible:");
    else
        SumOfSub (0, 1, sum);
}
```


}

void sumofsub (int m, int k, int r)

{

int i=1;

x[k]=1;

if (m + s[k] == d)

{

printf("subset :");

for (i=1; i<=k; i++)

if (x[i] == 1)

printf("\t %d", s[i]);

printf("\n");

}

else

if (m + s[k] + s[k+1] <= d)

sumofsub(m + s[k], k+1, r - s[k]);

if ((m + r - s[k] >= d) && (m + s[k+1] <= d))

{

x[k]=0;

sumofsub(m, k+1, r - s[k]);

}

}

Output: -

Enter the number of elements: 5001

Time taken for sorting 5001 elements: 0.04000

Enter the number of elements: 6000

Time taken for sorting 6000 elements: 0.060000

Enter the number of elements: 7000

Time taken for sorting 7000 elements: 0.074000

Enter the number of elements: 8000

Time taken for sorting 8000 elements: 0.099000

Enter the number of elements: 9000

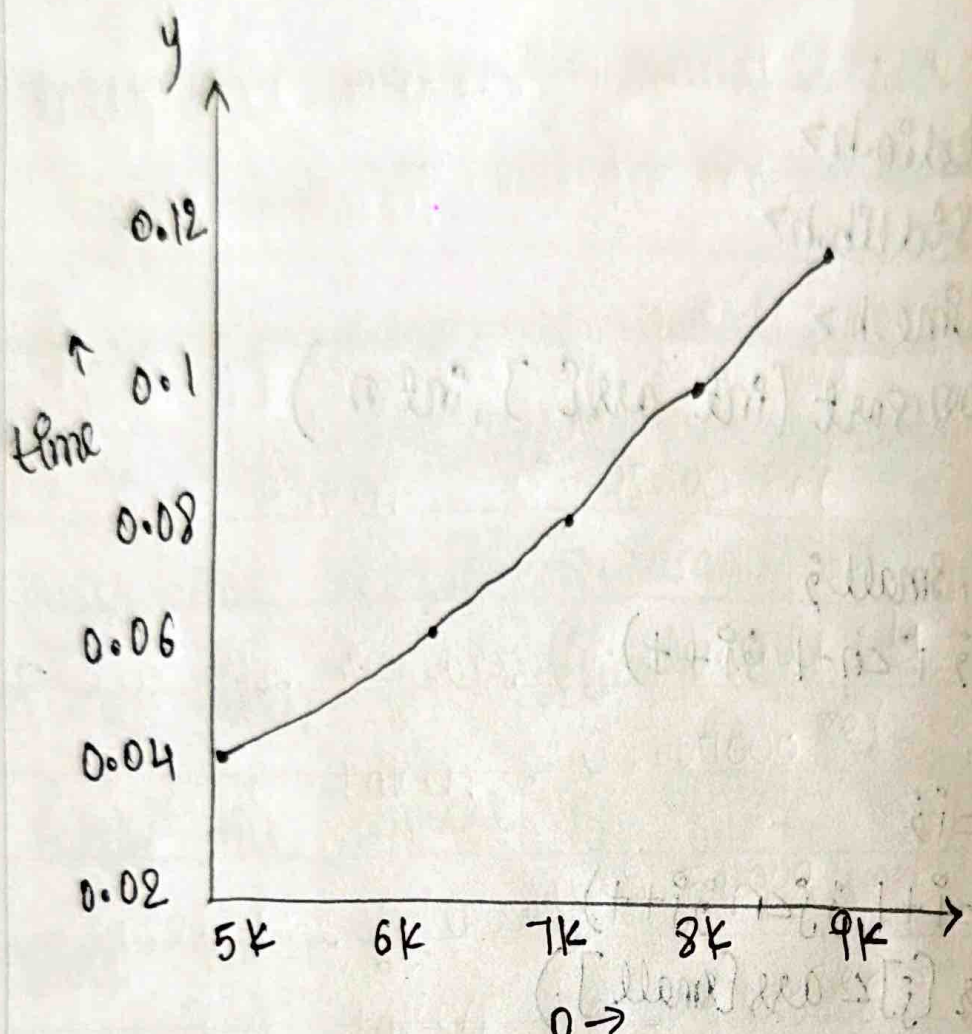
Time taken to sort 9000 elements: 0.124000

Program 9

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
void selectionSort (int arr[], int n)
{
    int p, j, small;
    for (i = 0; i < n - 1; i++)
    {
        small = i;
        for (j = i + 1; j < n; j++)
            if (arr[j] < arr[small])
                small = j;
        int temp = arr[small];
        arr[small] = arr[i];
        arr[i] = temp;
    }
}

int main()
{
    int n, a[1000], k;
    clock_t st, et;
    double ts;
```


time v/s n




```
printf("\n The Random numbers are : \n");  
for (k=1; k<=n; k++)  
{  
    a[k] = rand();  
    printf("%d\t", a[k]);  
}  
st = clock();  
selectionSort(a, n);  
et = clock();  
ts = (double)(et - st) / CLOCKS_PER_SEC;  
printf("\n Sorted numbers are : \n");  
for (k=1; k<=n; k++)  
{  
    printf("%d\t", a[k]);  
}  
printf("\n The time taken is %.e", ts);
```


Output

Enter the number of elements : 20000

time taken : 0.00300 sec

Enter the number of elements : 30000

time taken : 0.005000 sec

Enter the number of elements : 40000

time taken : 0.007000 sec

Enter the elements : 50000

time taken : 0.009008 sec

Program 10

#include <stdio.h>

#include <stdlib.h>

#include <time.h>

void ~~ex~~ Exch (int *p, int *q)

{

int temp = *p;

*p = *q;

*q = temp;

}

void QuickSort (int a[], int low, int high)

{

int i, j, key, k;

if (low >= high)

return;

key = low;

i = low + 1;

j = high;

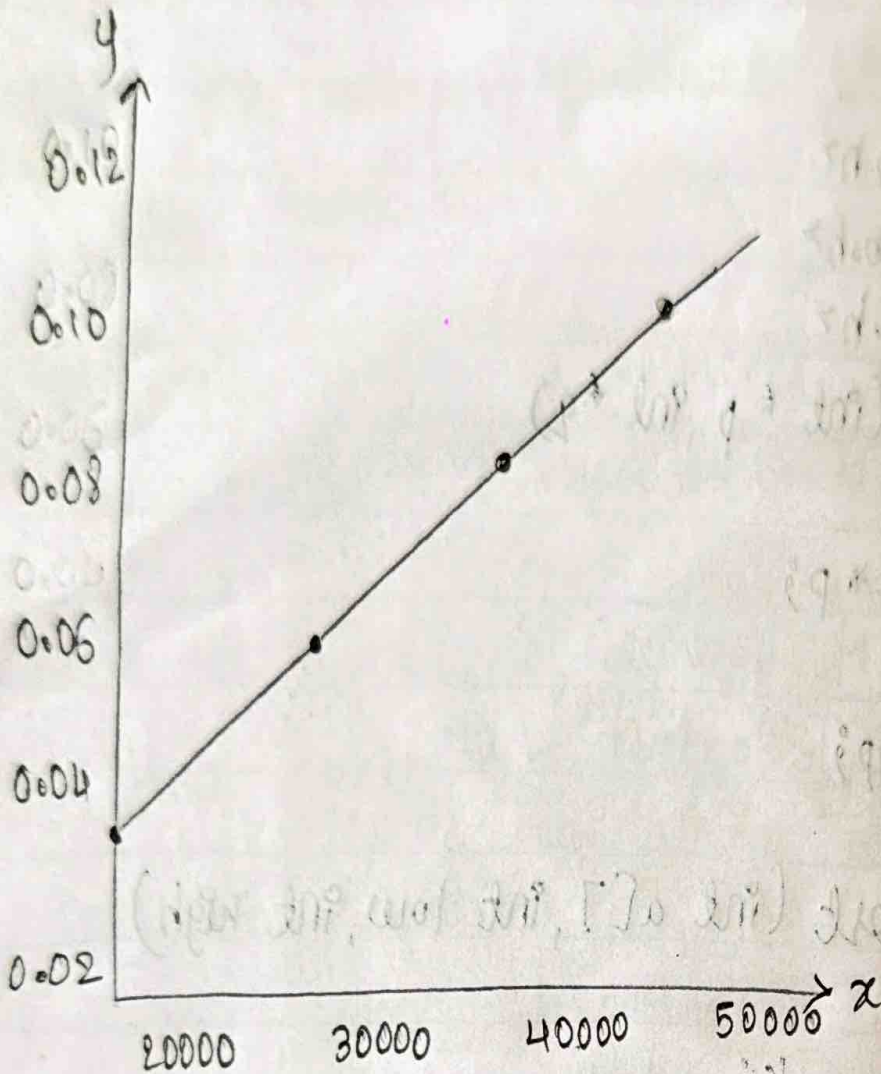
while (i <= j)

{

while (a[i] <= a[key]) i = i + 1;

while (a[j] > a[key]) j = j - 1;

if (i < j) Exch (&a[i], &a[j]);




```
}  
    exch(&a[j], &a[key]);  
    quicksort(a, low, j-1);  
    quicksort(a, j+1, high);  
}  
int main()  
{  
    int n, a[1000], k;  
    clock_t st, et;  
    double ts;  
    srand(time(NULL));  
    printf("Enter how many numbers:");  
    scanf("%d", &n);  
    printf("The random numbers are:\n");  
    for(k=1; k<=n; k++)  
    {  
        a[k] = rand();  
        printf("%d\t", a[k]);  
    }  
    st = clock();  
    quicksort(a, 1, n);  
    et = clock();  
    ts = (double)(et-st)/CLOCKS_PER_SEC;  
    printf("\n Sorted Numbers are:\n");
```



```
for (k=1; k<=n; k++)
```

```
print ("%d\t", a[i]);
```

```
printf("\n The time taken is %.e", ts);
```

}

Date:

Output

Enter the number of elements: 6000
time taken to sort 6000 elements: 0.000709
seconds

Enter the number of elements: 7000
time taken to sort 7000 elements: 0.000752
seconds

Enter the number of elements: 8000
time taken to sort 8000 elements: 0.000916

Enter the number of elements: 9000
time taken to sort 9000 elements: 0.001493

Enter the number of elements: 10000
time taken to sort 10000 elements: 0.001589

Program 11

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <time.h>
```

void merge (int a[], int low, int mid, int high)

```
int i, j, k, b[10];
```

$i = \text{low}, j = \text{mid} + 1; k = \text{low};$

```
while (i <= mid && j <= high)
```

if $(a \mid p] \subsetneq (a \mid i])$

$$b[k++] = a[i++] ;$$

else

$$b[k++] = a[j++];$$

```
while (p <= mid) b[k++] = a[l++];
```

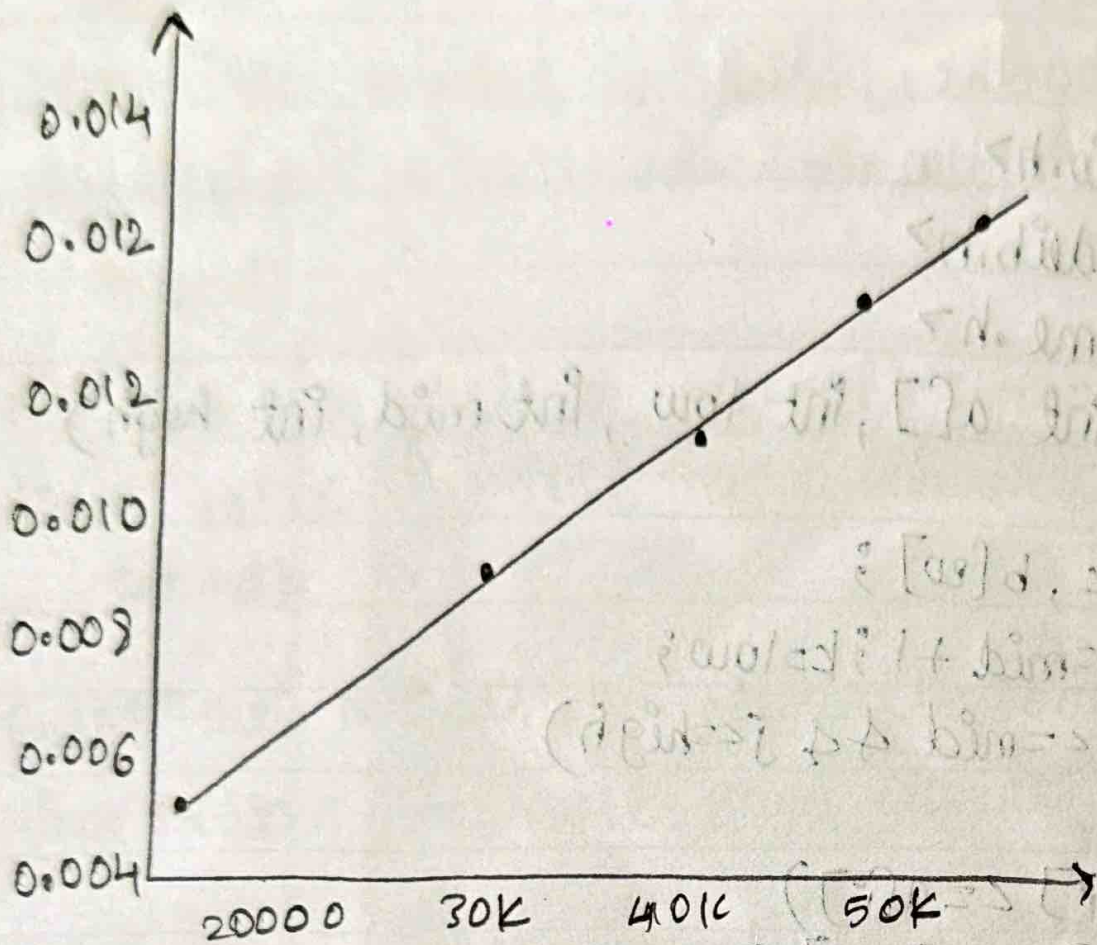
```
while (i <= high) b[k++] = a[j++];
```

for ($k = \text{low}$; $k = \text{high}$; $k++$)

$$a[k] = b[k];$$

void merge sort (int a[], int low, int high)

int mid;




```
if (low >= high)
    return;
mid = (low + high) / 2;
mergesort(a, low, mid);
mergesort(a, mid + 1, high);
merge(a, low, mid, high);
}
```

```
int main()
{
    int n, a[2000], k;
    clock_t st, et;
    double ts;
    srand(time(NULL));
    printf("\n Enter How many . numbers : ");
    scanf("%d", &n);
    printf("\n The Random Numbers are : \n");
    for (k = 1; k <= n; k++)
    {
        a[k] = rand();
        printf("%d\t", a[k]);
    }

    st = clock();
    mergesort(a, 1, n);
    et = clock();
}
```



```
ts = (double) (et-st) / CLOCK_S_PER_SEC;  
printf("\n Sorted Numbers are : \n");  
for (k=1; k<=n; k++)  
    printf("%d ", a[k]);  
printf("\n The time taken is %e", ts);  
}
```


Output:

Enter the number of queens

4

Solution 1

*	Q	*	*
*	*	*	Q
Q	*	*	*
*	*	Q	*

Solution 2

*	*	Q	*
Q	*	*	*
*	*	*	Q
*	Q	*	*

total solutions = 2

Program 12

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
int a[30], count = 0;
int place (int pos)
{
    int i;
    for (i = 1; i < pos; i++)
    {
        if (a[i] == a[pos] || (abs(a[i] - a[pos]) == abs(i - pos)))
            return 0;
    }
    return 1;
}

void print_sol (int n)
{
    int i, j;
    count++;
    printf ("In Solution # %d: \n", count);
    for (i = 1; i <= n; i++)
    {
```



```
{ for (j=1 ; j<=n ; j++)
```

```
    if (a[j] == j)
```

```
        printf ("%d\t");
```

```
    else
```

```
        printf ("%*\t");
```

```
    }
```

```
    printf ("\n");
```

```
}
```

```
void queen (int n)
```

```
{
```

```
    int k=1;
```

```
    a[k]=0;
```

```
    while (k!=0)
```

```
{
```

```
    a[k] = a[k] + 1;
```

```
    while (a[k] <= n) && ! place (k)
```

```
        a[k]++;
```

```
        if (k==n)
```

```
            print sol(n);
```

```
        else
```

```
        {
```

```
            k++;
```

```
            a[k]=0;
```



```
}  
}  
else
```

```
k--;
```

```
}
```

```
}  
int main()
```

```
{  
    int i, n;
```

```
    printf("Enter the number of Queens\n");
```

```
    scanf("%d", &n);
```

```
    queen(n);
```

```
    printf("\n Total solutions = %d", count);
```

atme

INDEX

Max. Marks.

Sl. No.	Experiments	Date	Page No.	Marks
01	Design & implement c/c++ program to find minimum cost spanning tree of a given connected undirected graph using Kruskal's algorithm	5/03/26	1 - 6	50
02	Design & implement c/c++ program to find minimum cost spanning tree of a given connected undirected graph using Prim's algorithm	12/03/26	7 - 10	50
03	a) Design & implement c/c++ program to solve all-pairs shortest path problem using Floyd's algorithm b) Design & implement c/c++ program to find the transitive closure using Warshall's algorithm.	26/03/26	11 - 14 15 - 19	50
04	Design & implement program to find shortest path from a given vertex in a weighted connected graph to other vertices using Dijkstra's algorithm	02/04/2026	19 - 24	50
05	C/C++ program to obtain the Topological ordering of vertices of given digraph		25 - 30	50
06	C/C++ program to solve 0/1 knapsack problem using dynamic programming method		31 - 36	50
07	C/C++ program to solve discrete knapsack & continuous knapsack problem using greedy approximation method		37 - 42	
08	C/C++ program to find the subset of given set $S = \{s_1, s_2, \dots, s_n\}$ of n positive integers whose sum is equal to given positive integer d		43 - 46	
09	C/C++ program to sort a given set of n integers elements using selection sort method & compare		47 - 50	

Cont'd.....

Sl. No.	Experiments	Date	Page No.	Mark
	its time complexity. Run the program for varied values of n = 5000 & record the time taken to sort.		PART	
10	C/C++ program to sort a given set of n integers elements using quick sort method & compute its time complexity. Run the program for varied values of n = 5000 & record the time		51-56	
11	C/C++ program to implement sort a given set of n integers elements using merge sort method & compute its time complexity. Run the program for varied values of n = 5000 & record the time		57-62	
12	Design and implement C/C++ program to N Queens problem using Backtracking		63-68	
	Average:			

Internal Lab Marks : _____ + _____ = _____

(LCR) (Test)

Signature of Staff